

MODULE 04 — ASSIGNMENT

Introduction to Programming Language with AI (JavaScript + Node.js)

Name: MD Jobael Haque (Arif) ID: 01675979838 Batch: 21

Course: AI Driven SQA — Manual & Automation Testing | OSTAD

Q1. JavaScript Coding Practice

The five programs below solve common programming problems in JavaScript. Each includes a problem statement, source code, verified output, and a short explanation of how the solution works.

1. Check Whether a Number Is Even or Odd

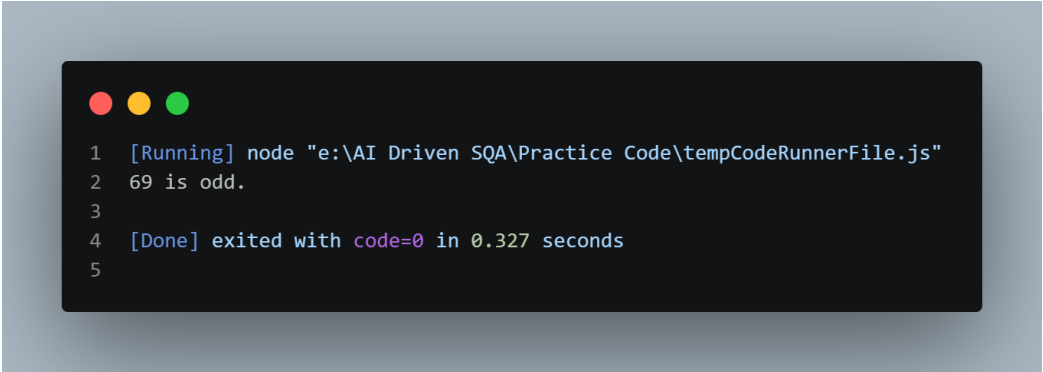
PROBLEM STATEMENT

Write a JavaScript program that checks whether a given number is even or odd.

SOURCE CODE

```
function checkEvenOdd(number) {  
  if (number % 2 === 0) {  
    return `${number} is even.`;  
  } else {  
    return `${number} is odd.`;  
  }  
}  
  
console.log(checkEvenOdd(69));
```

OUTPUT



```
1 [Running] node "e:\AI Driven SQA\Practice Code\tempCodeRunnerFile.js"  
2 69 is odd.  
3  
4 [Done] exited with code=0 in 0.327 seconds  
5
```

HANDS-ON PRACTICE

Executed live in VS Code to confirm the result independently.

The screenshot shows the VS Code interface with the 'Practice Code' workspace. The Explorer sidebar on the left shows a file named 'Assignment Module 4.js'. The main editor displays the following JavaScript code:

```

1 function checkEvenOdd(number) {
2     if (number % 2 === 0) {
3         return `${number} is even.`;
4     } else {
5         return `${number} is odd.`;
6     }
7 }
8
9 console.log(checkEvenOdd(69));

```

The Output panel at the bottom shows the execution result:

```

[Running] node "e:\AI Driven SQA\Practice Code\tempCodeRunnerFile.js"
69 is odd.

[Done] exited with code=0 in 0.327 seconds

```

EXPLANATION

The modulus operator (%) returns the remainder of a division. Dividing by 2 leaves a remainder of 0 for even numbers and 1 for odd numbers, so the if/else statement branches on that result. For 69 % 2, the remainder is 1, so the function returns "69 is odd."

2. Find the Largest of Three Numbers

PROBLEM STATEMENT

Write a JavaScript program that finds the largest value among three given numbers.

SOURCE CODE

```

function findLargest(a, b, c) {
    return Math.max(a, b, c);
}

console.log(findLargest(25, 72, 48));

```

OUTPUT

```

1 [Running] node "e:\AI Driven SQA\Practice Code\tempCodeRunnerFile.js"
2 72
3
4 [Done] exited with code=0 in 0.237 seconds
5

```

HANDS-ON PRACTICE

Executed live in VS Code to confirm the result independently.

```

2026 at 10:37 AM) ↔ logintest.js × JS LoginTestFeature.js × JS Assignment Module 4.js × ...
JS Assignment Module 4.js > ...
1
2 function findLargest(a, b, c) {
3   |   return Math.max(a, b, c);
4 }
5
6 console.log(findLargest(25, 72, 48));
7

Problems Output Debug Console Terminal Ports
[Running] node "e:\AI Driven SQA\Practice Code\tempCodeRunnerFile.js"
72

[Done] exited with code=0 in 0.237 seconds

```

EXPLANATION

Math.max() is a built-in method that accepts any number of arguments and returns the largest one. Passing 25, 72, and 48 directly avoids writing manual if/else comparisons and returns 72.

3. Reverse a String**PROBLEM STATEMENT**

Write a JavaScript program that reverses the characters of a given string.

SOURCE CODE

```

function reverseString(str) {
  return str.split("").reverse().join("");
}

console.log(reverseString("Sanjina Jaman"));

```

OUTPUT : namaJ anijnaS

```

1 [Running] node "e:\AI Driven SQA\Practice Code\Assignment Module 4.js"
2 namaJ anijnaS
3
4 [Done] exited with code=0 in 0.513 seconds
5
6

```

HANDS-ON PRACTICE

Executed live in VS Code to confirm the result independently.

The screenshot shows the VS Code interface with the 'Practice Code' workspace. The Explorer sidebar on the left shows a file named 'Assignment Module 4.js'. The main editor displays the following JavaScript code:

```

1 function reverseString(str) {
2   return str.split("").reverse().join("");
3 }
4
5 console.log(reverseString("Sanjina Jaman"));
6

```

The Output panel at the bottom shows the execution results:

```

[Running] node "e:\AI Driven SQA\Practice Code\tempCodeRunnerFile.js"
namaJ anijnaS

[Done] exited with code=0 in 0.231 seconds

```

EXPLANATION

`split("")` breaks the string into an array of individual characters, `.reverse()` flips the order of that array in place, and `join("")` stitches the characters back into a single string — turning "Sanjina Jaman" into "namaJ anijnaS".

4. Count Vowels in a String

PROBLEM STATEMENT

Write a JavaScript program that counts the number of vowels (a, e, i, o, u) in a string.

SOURCE CODE

```

function countVowels(str) {
  const vowels = "aeiou";
  let count = 0;

  for (const char of str.toLowerCase()) {
    if (vowels.includes(char)) count++;
  }
  return count;
}

console.log(countVowels("JavaScript Programming"));

```

OUTPUT

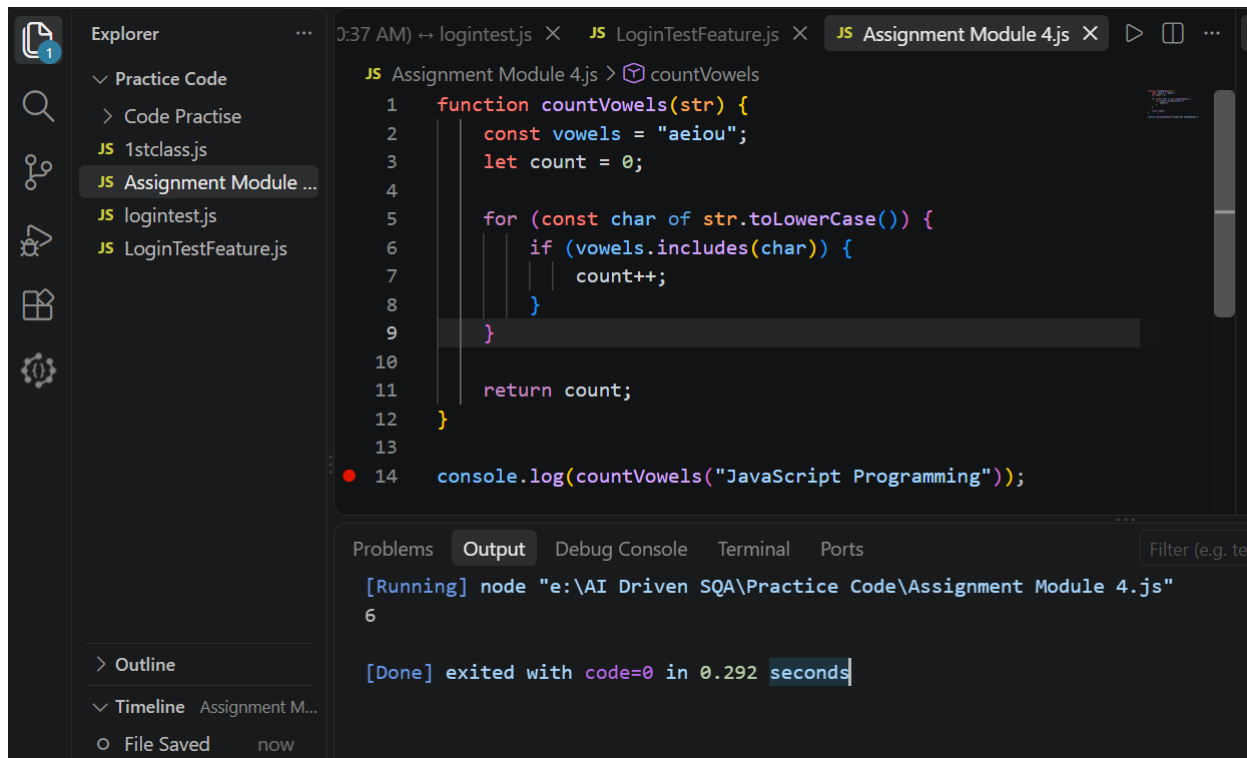
```

1 [Running] node "e:\AI Driven SQA\Practice Code\Assignment Module 4.js"
2 6
3
4 [Done] exited with code=0 in 0.194 seconds

```

HANDS-ON PRACTICE

Executed live in VS Code to confirm the result independently.



EXPLANATION

Converting the string to lowercase first means the check works regardless of the original letter case. The for...of loop inspects each character in turn, and vowels.includes(char) tests it against "aeiou". "JavaScript Programming" contains 6 vowels in total.

5. Remove Duplicate Values from an Array

PROBLEM STATEMENT

Write a JavaScript program that removes duplicate values from an array and returns only unique values.

SOURCE CODE

```

function removeDuplicates(arr) {
  return [...new Set(arr)];
}

console.log(removeDuplicates([10, 20, 10, 30, 20, 40]));

```

OUTPUT

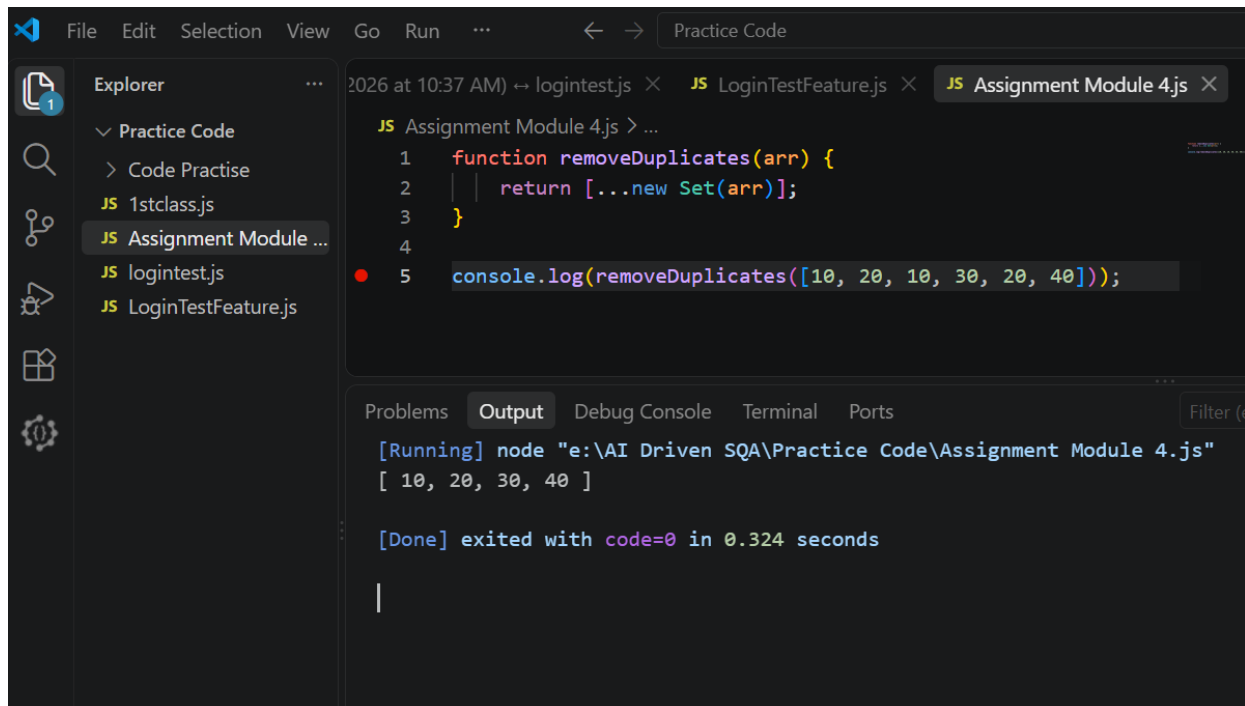
```

1 [Running] node "e:\AI Driven SQA\Practice Code\Assignment Module 4.js"
2 [ 10, 20, 30, 40 ]
3
4 [Done] exited with code=0 in 0.324 seconds

```

HANDS-ON PRACTICE

Executed live in VS Code to confirm the result independently.



EXPLANATION

A Set can only ever hold unique values — adding a duplicate simply has no effect. Wrapping the array in new Set(arr) drops the repeats automatically, and the spread operator (...) converts the Set back into a plain array: [10, 20, 30, 40].

Q2. Introduction to Node.js

2.1 What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime built on Google's V8 engine. It lets JavaScript run outside a web browser — on a server, a local machine, or inside a CI/CD pipeline — which is why it's widely used for backend services, command-line tools, APIs, and automation scripts. Because the same language now runs on both the front end and the back end, a QA engineer who already knows JavaScript can write automation code without switching languages.

2.2 How Does Node.js Work?

Node.js executes JavaScript through the V8 engine and layers an event-driven, non-blocking I/O model on top of it. Instead of pausing the whole program while it waits on something slow — reading a file, querying a database, calling an API — Node.js hands that task off and keeps running the rest of the code. When the slow task finishes, a callback (or Promise) picks up the result. This is what lets a single Node.js process handle many concurrent operations efficiently, which matters for automation work that's often just waiting on requests, files, or subprocesses.

Simple flow: Script runs → Node.js hands off I/O to the system → the event loop keeps executing other code → a callback returns the result once it's ready.

2.3 Advantages of Node.js for QA Automation

Advantage	Relevance to QA Automation
One language, front to back	Testers who already know JavaScript can write automation scripts without learning a second language.
npm ecosystem	A huge library of ready-made packages covers test runners, HTTP clients, reporting, and data handling.
Strong tooling support	Most modern browser- and API-automation frameworks (Playwright, Cypress, Jest, etc.) run on Node.js.
Asynchronous by design	Well suited to automation that waits on network calls, file I/O, and API responses.
Command-line friendly	Test suites run easily from the terminal and slot naturally into CI/CD pipelines.
Cross-platform	The same scripts run on Windows, macOS, and Linux test environments without changes.

2.4 Browser JavaScript vs. Node.js

Aspect	Browser JavaScript	Node.js
Where it runs	Inside a web browser	Outside the browser, via the Node.js runtime
Main purpose	Adds behaviour/interactivity to web pages	Builds servers, APIs, scripts, and automation tools
DOM access	Can read/manipulate the page's DOM directly	No DOM by default — there's no page to manipulate
File system access	Restricted by browser security	Full file-system access via built-in modules
Global objects	window, document	global, process, module
Package management	Not managed by Node.js/npm	Managed through npm and the Node ecosystem
Typical automation use	Code that runs as part of a web page	Running test scripts, automation frameworks, and CI jobs

Conclusion: JavaScript provides the language foundation for the coding exercises in this module, while Node.js extends that same language beyond the browser. For QA automation specifically, Node.js matters because it can run test scripts and tooling from the command line, work directly with files and APIs, pull in npm packages for almost any testing need, and fit naturally into an automated CI/CD pipeline.